BTW GLOBAL DISTRIBUTORS
YOUR GLOBAL SUPPLY PARTNER

SUPPLY CHAIN, LOGISTICS, INVENTORY & SALES PLATFORM

Platform architecture & processes

What has been built so far: the system's structure, how a consignment moves from plan to reconciliation, how data flows, who uses what, and the data kept by each module. Version 1.2 · 25 October 2026 · source of truth: the code in this repository.

New in 1.2. The client's real roster (Daniel, Michael, Kabonge, Testfie, Joel, Eric, Moses, Asie, Yonas, drivers) with an access matrix scoped by company, **buyer–supplier pair**, loading point, warehouse and route; loaders, receivers and drivers see **no prices**, and Testfie records the agreed transport price **after loading**. **Warehouse money** (expenses, stock counts, re-exports Asie → Yonas, transport-fee ageing, selling prices), **landed cost v2** and the combined performance report; 31 dashboards behind one endpoint `GET /v1/dashboards`, with a home screen per person on the web and the phone. Migrations `0009` and `0010`. POs are issued without an approval step (still a switch in `config.toml`, off).

3
apps on one server: office web, phone app (field roles + office team PWA), API

9
named people + drivers, each scoped by company, pair, loading point or warehouse

39
database tables (PostgreSQL + PostGIS)

6
integrations behind adapters: SMS/USSD, email, push, files, PDF, routing

Contents

1. Architecture diagram
2. Architecture explained
3. Process diagram (by actor)
4. Process & reconciliations explained
5. People, access & landed cost v2
6. Data flow diagram
7. Use case diagram
8. Data structures per module (2 pages)

How to read the diagrams

Request / main flow — moving navy arrow (a call, a step done in order)

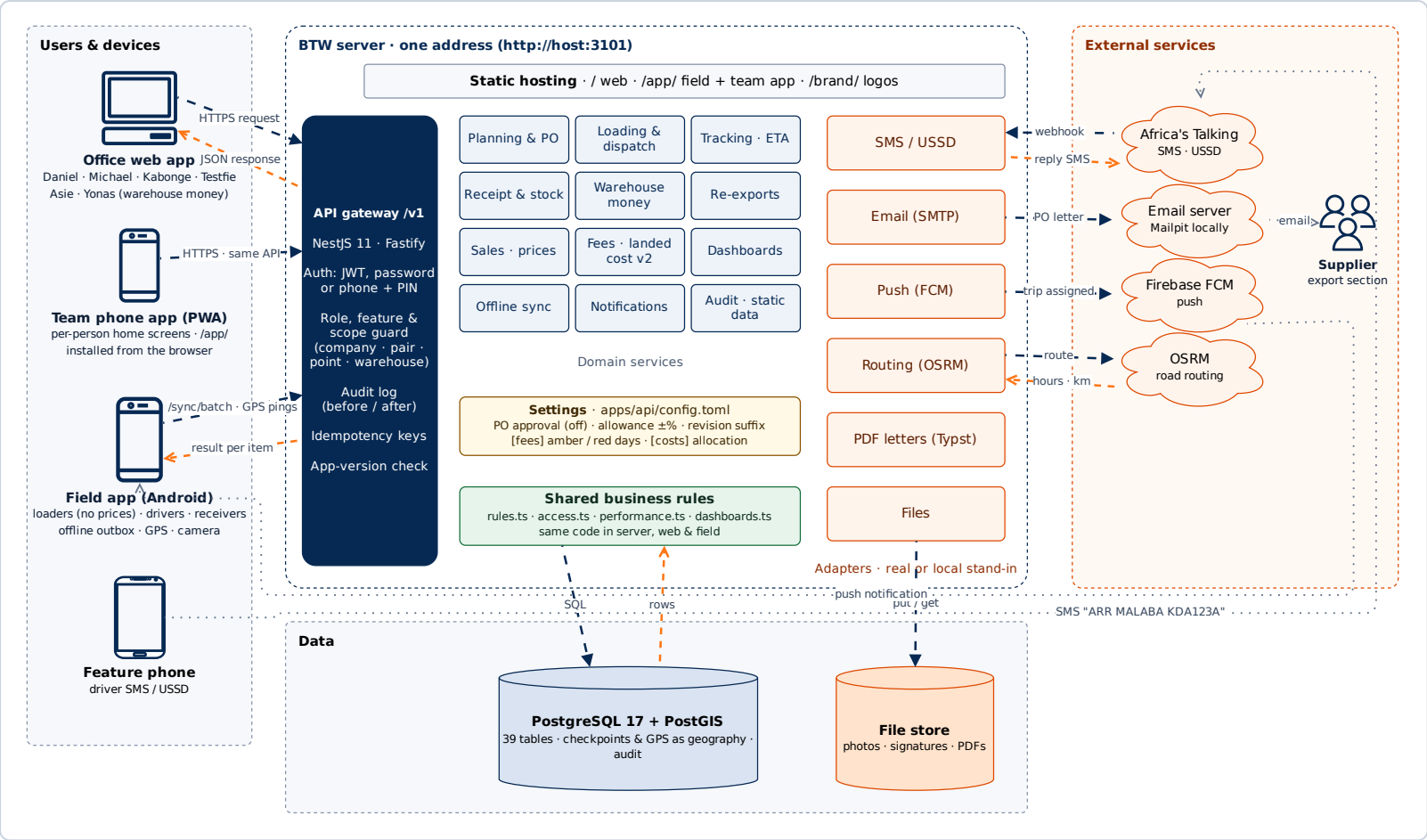
Response — moving dashed orange arrow (data returned)

Hand-over / notification — moving dotted arrow (between people, push, SMS, email)

The arrows move on screen (this HTML file, or the `.drawio` files opened in draw.io / diagrams.net, where each diagram can be edited). In print or PDF they appear as static dashed lines. Click a diagram to open it full screen.

One server, light apps, swappable integrations

diagrams/01-architecture.drawio



Clients. Office staff use the web app, or the team phone app (PWA) at `/app/`; the three field roles use the Android app, which queues work offline and sends it in one batch. Drivers without data can check in by SMS or USSD.

Server. One NestJS process serves the apps and the API at the same address. Every request passes auth, role, feature and scope checks (company, pair, loading point, warehouse) and is written to the audit log before a domain service runs the shared business rules. Business switches and thresholds come from `config.toml`.

Data & integrations. PostgreSQL/PostGIS holds all operational data, including checkpoints and GPS. Each external service sits behind an adapter: without a key it uses a local stand-in, so nothing changes in code when keys are added.

1 · ARCHITECTURE

Architecture explained

Layers

LAYER	WHAT IT IS	TECHNOLOGY
Office web app	Home per person, 31 dashboards (PDF / Excel / CSV), plans, POs, queues, Truck desk, warehouse money, re-exports, fees, selling prices, static data, users, audit log.	Svelte 5 + Vite · 100.3 KB first load · lazy charts
Field app	One action at a time: *Hand over* (±allowance, no prices) · *I've arrived* · *Confirm receipt* (loss reasons) · expenses, stock count, re-export. Offline outbox, "last synced".	Svelte 5 + Capacitor 7 · 79.4 KB
Team phone app (PWA)	Office staff at `/app/`: a home per person (directors' scorecard, Kabonge's pipeline, Testfie's fleet) with one-list drill-downs. Installable from the browser.	`apps/field/src/team`
API gateway	Sign-in (password or phone + PIN), refresh tokens, role, feature & scope guard (company, pair, loading point, warehouse, route), audit with before / after, idempotency keys.	NestJS 11 on Fastify · JWT · bcrypt
Domain services	Planning & PO, logistics, stock, warehouse money, re-exports, sales, fees, performance, dashboards, notifications, sync. Rules shared with the apps.	TypeScript · `packages/api/src/rules`, `access`, `performance`, `dashboards`
Data	39 tables; nested items (plan lines, passes, issues, ack lines) as JSON in their parent record; GPS and geofences as PostGIS geography.	PostgreSQL 17 + PostGIS 3.5 · Drizzle ORM
Integrations	SMS/USSD (Africa's Talking), email, push (Firebase), files, PDF letters (Typst), routing (OSRM); each with a local stand-in.	Adapters selected from `.env`
Business settings	PO approval (off), allowance ±10%, revision suffix, `[fees]` 14 / 30 days, `[costs] allocation`. Sent in `GET /v1/ref`.	`apps/api/config.toml` (or `CONFIG_FILE`)

Life of a request — a driver's check-in

1. The driver slides *I've arrived at Malaba*; the app stores it in the outbox with a unique key and, when online, sends `POST /v1/sync/batch`.
2. The gateway checks the token, role and that the leg belongs to this driver, audits the call, and the idempotency check applies a retried item once only.
3. The tracking service records the pass, recalculates leg status and ETA, and raises alerts.
4. A result per item comes back; the app clears the outbox. Office screens see the change on their next refresh.

Running it

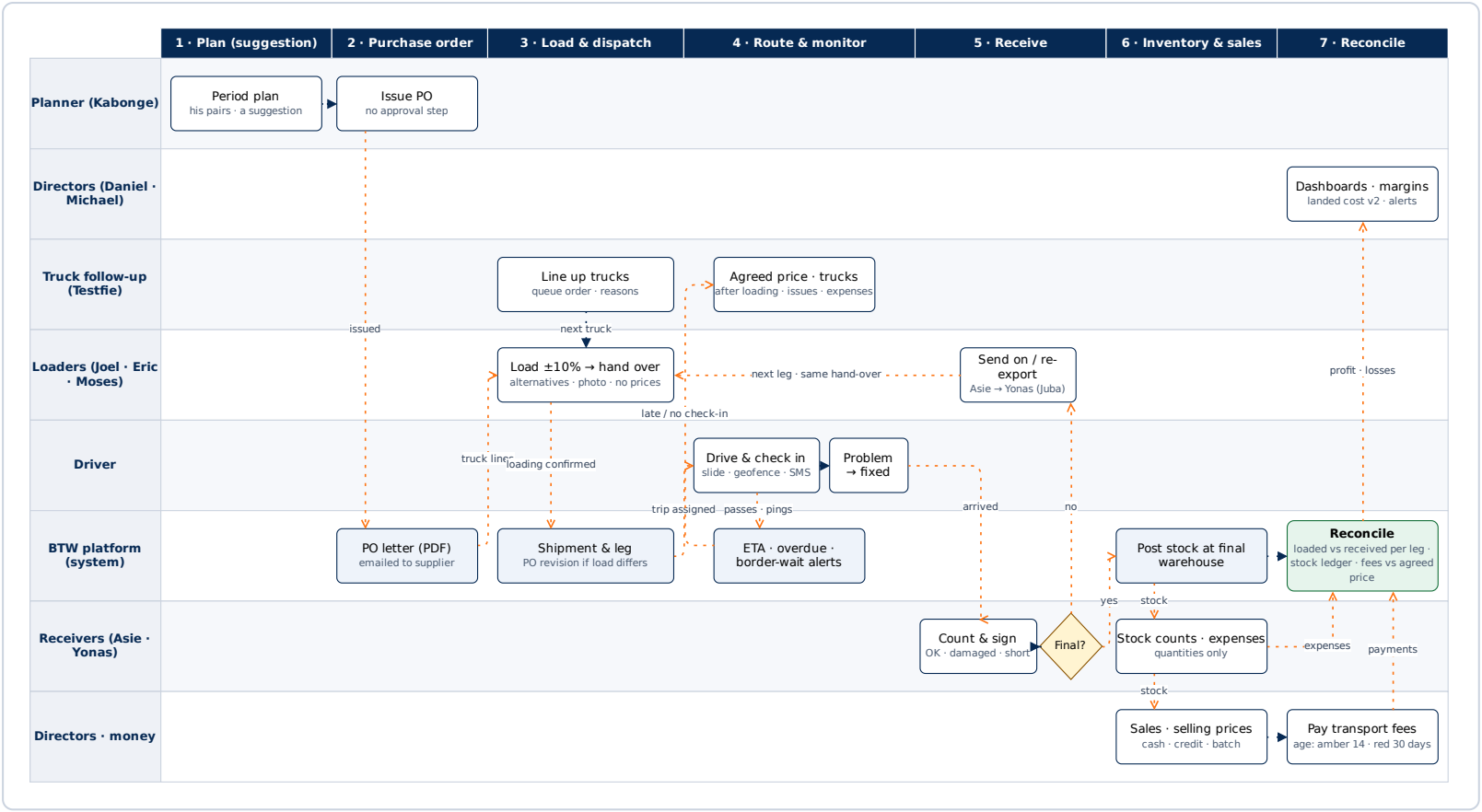
`pnpm start` starts the database and email inbox (Docker), migrates, seeds demo data if empty, builds both apps and serves everything from **http://localhost:3101**: `/` web, `/app/` field, `/brand/` logos, `/v1` API.

Quality gates. `pnpm verify` (type-check, workflow test, API test, builds, size budget) · `test:qa` 1,108 API, access-matrix, money, dashboard, security and integration checks (all passing) · `pnpm e2e` browser journeys with the named users.

2 · PROCESS

From planning to reconciliation, by actor

diagrams/02-process.drawio



Read left to right. Each lane is a person or the system. Solid moving arrows are steps one person does in order; dotted orange moving arrows are hand-overs and notifications between people. Kabonge's PO is issued at once (no approval step). The diamond is the receiver's "final destination?", which either posts stock or sends the goods on — at Gisenyi Asie raises and loads a re-export to Yonas in Juba. Loaders hand over without prices; Testfie records the agreed price after loading, follows every truck and its costs; the directors set selling prices, pay fees and read the dashboards.

2 · PROCESS

Process steps and reconciliations

PHASE	WHO	WHAT HAPPENS	SYSTEM RECORD & CONTROL
1 Period plan	Kabonge	A suggestion per company and period; he edits only his pairs' lines. No approval; each save is a version. It gates nothing.	Active / Replaced; no overlapping Active plans (409 → “Replace?”).
2 Purchase order	Kabonge (BTW–KBL, Nihna–UBL, Kivu–SBL); directors (Nihna–GBI)	Fills truck lines and taps *Issue purchase order*. Letter on the company letterhead, emailed to the supplier.	Ref per company and year (BTW-PO-2026-0002); “ISSUED” stamp; no approval step (switch off).
3 Load & dispatch	Joel, Eric (KBL) · Moses (UBL) · Testfie (GBI) · Testfie after loading	Testfie and Kabonge order the queue. The loader loads from the PO within ±10%, may load alternatives, adds photos and slides *Hand over* — no prices. Testfie then records the agreed transport price.	Any difference → PO revision `…-rev001` (auto-issued, notified); shipments follow the latest revision.
4 Route & monitor	Driver · Testfie	Driver checks in at each checkpoint (slide, automatic geofence, or SMS/USSD), reports problems and "fixed".	ETA early / on time / late; overdue and border-wait alerts.
5 Receive	Asie (Gisenyi, Goma) · Yonas (Juba)	"Everything arrived OK", or adjust received / damaged per brand with a reason and photos; signs. Asie re-exports Rwanda → Juba as a new shipment `ASIE-RX-…`.	Discrepancy alert; loss reasons feed the Pareto; Yonas acknowledges re-exports.
6 Inventory & sales	System · Asie, Yonas · directors	Stock posts at the warehouse after acknowledgement; receivers record expenses and stock counts (quantities only); directors sell at their selling prices.	Ledger per warehouse and SKU; sale above stock blocked unless a director overrides.
7 Reconcile	Directors · Testfie (transport side)	Transport fees paid against references; landed cost v2, profit and margins at product, warehouse, company and business level.	Fee age flags; combined performance report; dashboards and alerts.

Reconciliations the platform performs

CHECK	RULE	WHERE IT SHOWS
Ordered vs loaded	Truck total within `±allowance_pct` of the PO and never above capacity; otherwise refused. A difference creates the next revision, numbered from the original PO; superseded orders are not counted twice.	Allowance meter at hand-over, PO revision chain
Loaded vs received, per leg	Each leg: loaded − received = short; damaged counted separately; the next leg carries only good cases forward, so a loss is pinned to the leg where it happened.	Shipment reconciliation table, discrepancy alert
Stock ledger	Opening + received + transfers in − transfers out − damaged ± adjustments − sold = closing; in-transit shown separately.	Inventory (with ledger per SKU)
Transport fees	Agreed price (recorded by Testfie after loading) vs payments with status Paid; age from loading, amber at 14 and red at 30 days while unpaid or part paid.	Transport fees, unpaid & ageing dashboard
Landed cost v2	Goods + every trip cost of the shipment + the warehouse's share of its month's expenses, ÷ good cases received (page 6).	Performance report, landed cost dashboards
Offline replay	Each phone action has one key; replays apply once and return the stored result.	Sync screen, audit log

2 · PROCESS

People, access and the money they record

PERSON	ROLE ID	DOES	SCOPE · PRICES
Daniel, Michael	`director`	Users, static data, selling prices, payments, sales, performance, audit log	Every company (Group view) · all
Kabonge	`planner`	Period plans, POs, queue order on his points, providers / trucks / drivers and static data	Pairs BTW–KBL, Nihna–UBL, Kivu–SBL · PO prices
Testfie	`truck_planner` (truck follow-up officer)	Every truck, queue order, trip expenses, agreed price after loading, fees; loads at GBI Mombasa	All trucks · transport side only
Joel, Eric · Moses	`loader`	Waiting list, load ±10%, hand over, today's list	KBL Nairobi · UBL Kampala · no prices
Asie	`receiver` + `reexport`	Receive and sign, warehouse expenses, stock counts, re-export orders to Juba	Gisenyi, Goma · no purchase prices
Yonas	`receiver`	Receive and sign (incl. Asie's re-exports), warehouse expenses, stock counts	Juba · no purchase prices
Drivers	`driver`	Own trip: check in, report a problem (app or SMS)	Own trips · nothing financial

New modules in 1.2

MODULE	WHAT IT DOES	WHERE
Access scopes	Feature keys per role and the "Own" rules (pairs, points, warehouses, routes); prices stripped for loaders, receivers, drivers.	`packages/api/src/access.ts`, `rules.ts`
Warehouse money	Expenses (USD at month rate), stock counts, fees and ageing, selling prices, performance.	`/v1/warehouse/*`, `/finance/fees`, `/selling-prices`, `/reports/performance`
Re-exports	Asie raises and loads; new shipment Rwanda → Juba; Yonas acknowledges.	`/v1/reexports`, `/:id/load`, `/:id/cancel`
Agreed price	Testfie records it after loading; "not recorded" is alerted.	`POST /v1/shipments/:id/agreed-price`
Dashboards	31 dashboards; cut to the caller's scope, then filtered; home set per person.	`GET /v1/dashboards`, `/:key` · `dashboards.ts`
Audit & static data	Audit log with before / after; static data editable with history.	`/v1/audit`, `/v1/masterdata/*`

Data flow — landed cost v2 packages/api/src/performance.ts

Goods
cases loaded × PO price
`loadings · pos`

+

Truck
every trip cost of the shipment, all legs, split by cases
`trip_costs`

+

Warehouse share
month's expenses ÷ good cases received that month × the line's good cases
`warehouse_expenses · acks`

÷

Quantity received
received − damaged
`acks`

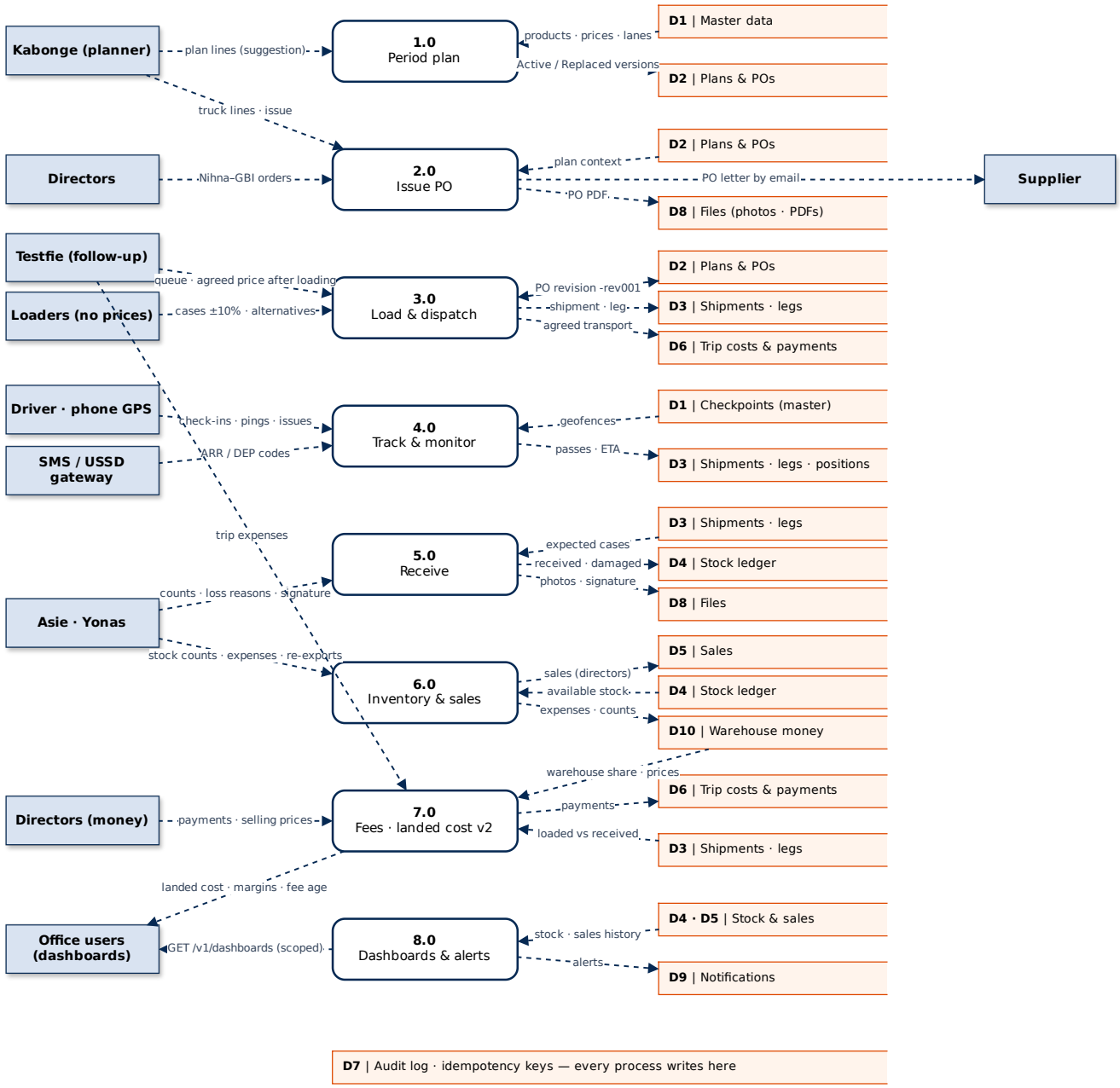
=

Landed cost per case
× cases sold = cost of units sold
`sales`

Profit = revenue − cost of units sold · **margin** = profit ÷ revenue · **price for a target margin** = landed cost per case ÷ (1 − target). Goods sent on or re-exported carry the landed cost per case of the warehouse they left. Split by cases (`[costs] allocation = "quantity"`) or by value. Example: $12.98 / case, 600 sold at $15 → profit $1,212.18, margin 13.47 %.

Data flow diagram (level 1)

diagrams/03-data-flow.drawio



Boxes (blue) are people and outside systems that send or receive data.

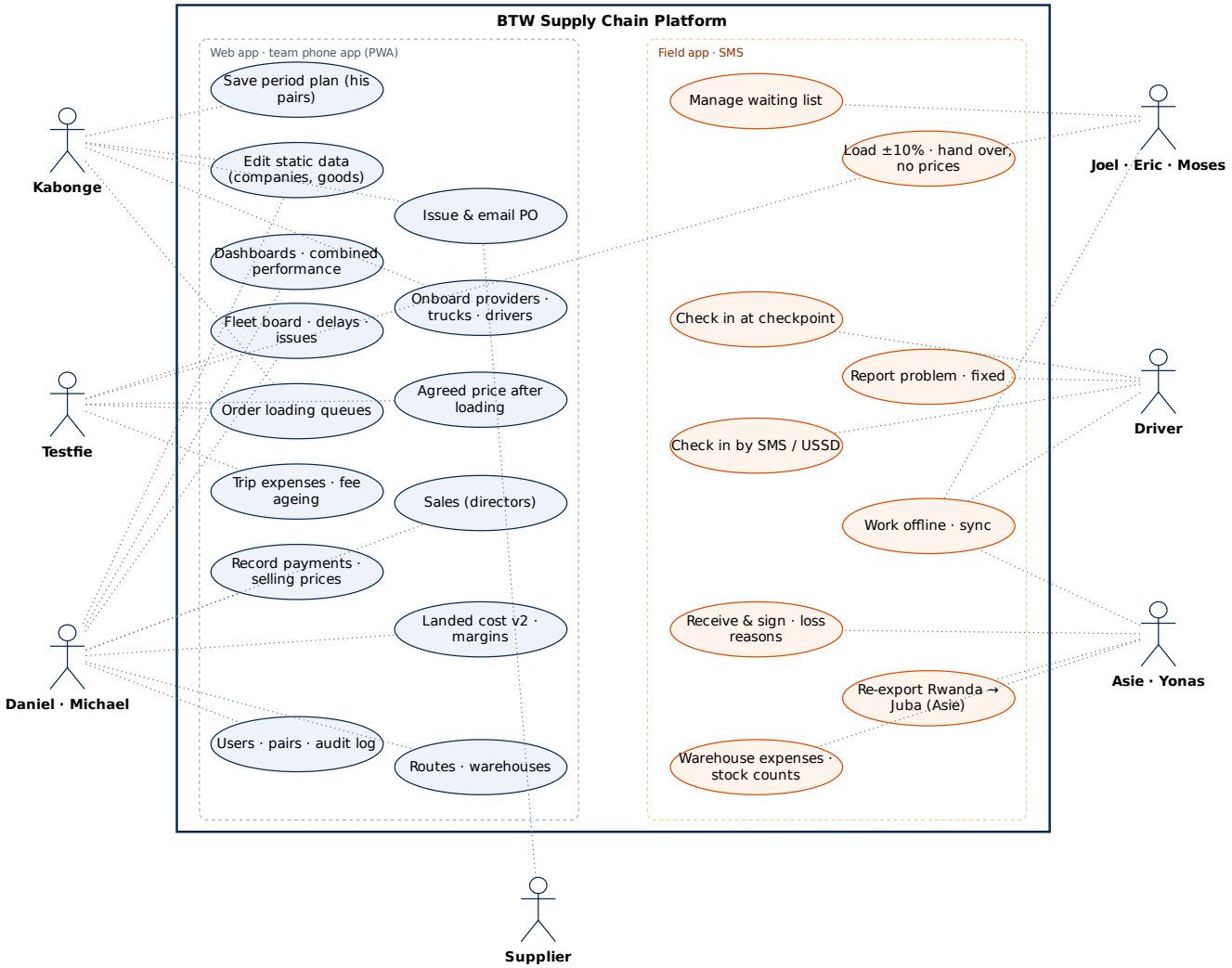
Rounded boxes 1.0–8.0 are the platform's processes; each maps to a service in the API.

Open boxes D1–D10 are data stores; the tables behind each are listed on pages 9–10.

4 · USE CASES

Who does what

diagrams/04-use-cases.drawio



Office roles (left) work in the web app or the team phone app (PWA). **Field roles** (right) use one task-focused screen each in the field app; drivers without data use SMS or USSD. Every action is limited to the person's scope: companies, buyer–supplier pairs (Kabonge), loading points (loaders; Testfie at GBI Mombasa), warehouses (Asie, Yonas) and, optionally, routes. Directors set up users, pairs and static data and read the audit log.

5 · DATA STRUCTURES

Data kept per module

Tables in PostgreSQL (`apps/api/src/db/schema.ts`). *Italic* fields are JSON lists stored inside the record.

Identity & access D1, D7

TABLE	KEY FIELDS
`users`	id, username, name, phone, password hash, PIN hash, roles[], entity_ids[], loading_point_ids[], warehouse_ids[], **pairs** (company–supplier), **route_ids[]**, driver_id, active, **last_login_at**, token_version
`devices` · `refresh_tokens`	device id, user, push token, platform, app version, last seen, revoked at · token hash, device, expiry, revoked at

Master data D1

TABLE	KEY FIELDS
`entities`	code (BTW, NUD, KVD), name, address, PO prefix, **active** (also on suppliers, products, receiving companies)
`receiving_companies` · `warehouses`	Asie, Yonas, countries · warehouse name, receiving company, checkpoint
`suppliers` · `products` · `price_list`	code (KBL, UBL, SBL, GBI), city, email, checkpoint · SKU, brand, pack (24 × 330 ml), units per case · price, currency, effective from
`providers` · `trucks` · `drivers`	name, kind (company / sacco / individual / market), contact, email, notes, regular, active · registration, trailer, capacity in cases, default driver, documents, active · name, phone, licence, has app, active
`checkpoints` · `routes`	name, kind (loading, weighbridge, border, warehouse), code (MALABA), lat/lng, geofence radius, geography · lane code, distance, *stops: checkpoint, planned hours, planned dwell*

Planning & purchase orders D2

TABLE	KEY FIELDS
`period_plans` · `period_plan_versions`	ref, entity, period kind, from / to, status (Active / Replaced), replaced by, version, *lines*, *history* · snapshot of every save
`plans` · `plan_versions`	PO order: ref, entity, supplier, status (Draft → Submitted → Approved / Issued), version, *lines: truck no, product, cases, price, to, route*, *history*, revision of / no., superseded by · snapshots
`pos`	ref, plan, entity, supplier, approved / issued by and at, signature, approval (director / issued / revision), revision of / no., *changes: truck, product, ordered → loaded, kind*, reason, on behalf of, superseded by

Loading & logistics D3

TABLE	KEY FIELDS
`waiting_entries`	loading point, truck, driver, arrived at, position, status (Waiting / Assigned / Loaded)
`loadings`	PO, truck no, truck, driver, loading point, loaded at, *lines: ordered vs loaded cases*, agreed transport (empty until Testfie records it) + currency, *documents*, status
`shipments`	ref (SHP-2026-0041), PO, truck no, entity, loading point, receiving company, final warehouse, status (derived from legs)
`legs`	shipment, seq, route, from / to, truck, driver, status, departed, planned arrival, *passes: checkpoint, planned, arrived, departed, source*, last ping, *lines*, *issues: place, reason, fixed at, new due date*, acknowledgement
`positions`	leg, truck, lat/lng, geography, time, source (app / GPS provider), accuracy, speed

Data kept per module (continued)

Receipt & inventory D4, D8

TABLE	KEY FIELDS
`acks`	leg, shipment, warehouse, receiver name, time, *lines: product, expected, received, damaged,* ***reason****, remarks*, signature, *photos*, has discrepancy, by user
`stock_moves`	warehouse, product, kind (received, damaged, sold, transfer in / out, adjustment, moved out), cases, time, reference, **reason**, **note**, **by user** — the ledger behind every stock figure; quantities only

Warehouse money & re-exports D10 · migration 0010

TABLE	KEY FIELDS
`warehouse_expenses`	warehouse, receiving company, date, category (offloading, storage, salary, rent, utilities, damaged goods, other), amount, currency, fx rate, amount USD, reference, note, receipt, by user
`selling_prices`	company, product, warehouse (empty = company default), price, currency (USD), effective from, note, set by
`reexport_orders`	ref (ASIE-RX-2026-0001), company, from / to warehouse, route, *lines*, status, truck, driver, shipment, loaded by / at, delivered at, follow-up by

Sales D5

TABLE	KEY FIELDS
`sales`	ref, time, warehouse, customer, salesperson, kind (cash / credit / batch), *lines: product, cases, price*, currency, paid, status (Paid / Part-paid / Unpaid)

Finance & landing cost D6

TABLE	KEY FIELDS
`trip_costs`	shipment, leg, kind (transport, taxes & duties, clearing & forwarding, loading & offloading, storage, other), amount, currency, document ref
`trip_payments`	shipment, cost line, amount, time, reference (M-Pesa / bank), status (Pending approval / Approved / Paid), approved by, receipt file

Platform: sync, audit, notifications, files D7, D8, D9

TABLE	KEY FIELDS
`idempotency_keys`	key, user, operation, status (applied / duplicate / rejected), stored response — makes offline replays safe
`audit_log`	time, user, channel (web / app / sms), method, path, status, detail, **feature**, **record type / id**, **summary**, **before / after** (migration 0009)
`notifications` · `notification_reads`	title, body, link, for roles or for one user · who has read it
`files` · `settings`	name, type, size, url, uploaded by (photos, signatures, PDFs) · thresholds and other director settings

Not built yet

Receiving-company accounting (scope to confirm), P2 / P3 dashboards, one truck across several POs, the KBL receipt, alerts closed by hand, a reporting store for heavy dashboards, live SMS and push keys.